

Part 1: Basic Biodata Handling (Strong Foundation)

Use this sample dataset:

```
biodata = [  
    {"id": 1, "name": "Rahul", "age": 22, "gender": "Male", "city":  
    "Lucknow", "salary": 25000},  
    {"id": 2, "name": "Amit", "age": 25, "gender": "Male", "city": "Delhi",  
    "salary": 30000},  
    {"id": 3, "name": "Neha", "age": 21, "gender": "Female", "city":  
    "Lucknow", "salary": 28000},  
    {"id": 4, "name": "Priya", "age": 23, "gender": "Female", "city":  
    "Delhi", "salary": 26000}  
]
```

Level 1 – Basic Understanding

1. Print all names.
2. Find average age.
3. Find total salary of all employees.
4. Find the highest salary person.
5. Count male and female employees.
6. Print employees from Lucknow.
7. Print employees whose salary > 26000.
8. Add a new employee record.
9. Remove employee by ID.
10. Update salary of a specific employee.

Part 2: Filtering & Aggregation Logic

11. Count employees city-wise.
12. Find average salary city-wise.
13. Find youngest and oldest employee.
14. Create a new list containing only names and salaries.
15. Increase salary by 10% for all employees.
16. Print employees between age 21–23.
17. Sort employees by salary (ascending & descending).
18. Find employees whose name starts with "R".
19. Create age groups:
 - 20–22 → Group A
 - 23–25 → Group B
20. Calculate percentage of female employees.

Part 3: Data Cleaning & Real Analytics Thinking

21. If salary is `None`, replace it with average salary.
22. Remove duplicate records (based on ID).
23. Create a function to search employee by name.
24. Create a summary report:
 - Total employees

- Average age
- Average salary
- Highest paid employee
- Gender ratio

25. Normalize salary using:

$$(value - min) / (max - min)$$

26. Find outliers in salary

(Hint: salary > mean + 2 * std).

27. Convert dataset into:

- Separate lists (names, ages, salaries)

28. Export data to CSV file using file handling.

29. Create a menu-driven Biodata Analysis System:

- Add employee
- Delete employee
- Search
- Show report
- Exit

30. Create a simple text-based dashboard summary output.

Basic Sequence Analysis (Using Biopython)

Use:

```
from Bio.Seq import Seq
```

Level 1 – Basic DNA Operations

1. Create a DNA sequence using `Seq()` and print it.
2. Find the length of a DNA sequence.
3. Find GC content (percentage of G and C).
4. Convert DNA to RNA.
5. Find the complement of a DNA sequence.
6. Find the reverse complement.
7. Count number of A, T, G, C in a sequence.
8. Check whether a sequence contains a specific motif (e.g., "ATG").
9. Count how many times a motif appears.
10. Slice a sequence (print 5th to 15th base).

Part 2: Protein & Translation

11. Translate a DNA sequence into protein.
12. Count number of amino acids in protein sequence.
13. Find if protein contains stop codon (*).
14. Count frequency of each amino acid.
15. Extract first 10 amino acids from translated protein.

16. Check if sequence starts with start codon (ATG).
17. Identify all possible start codon positions.
18. Translate only a specific region of DNA.
19. Remove stop codon from protein sequence.
20. Compare two DNA sequences and count mismatches.

Part 3: Working with FASTA Files

Use:

```
from Bio import SeqIO
```

21. Read a FASTA file and print all sequence IDs.
22. Print sequence length for each record.
23. Find longest sequence in FASTA file.
24. Calculate GC content for each sequence.
25. Filter sequences longer than 100 bases.
26. Count total number of sequences in file.
27. Save filtered sequences into new FASTA file.
28. Extract only sequences containing a specific motif.
29. Convert all sequences to RNA and save.
30. Create summary report:

- Total sequences
- Average length
- Highest GC content
- Lowest GC content

Part 4: Analytical & Logical Thinking

31. Identify sequences with GC content > 60%.
32. Normalize sequence lengths.
33. Find duplicate sequences.
34. Create function to calculate GC content.
35. Create function to find mutation positions between two sequences.
36. Create function to calculate AT/GC ratio.
37. Detect palindromic sequence.
38. Identify open reading frames (basic logic).
39. Count codon frequency.
40. Create a mini DNA analysis tool with menu:

- Input sequence
- GC content
- Complement
- Reverse complement
- Translate
- Exit

Numpy Practice Gene Expression Analysis

Dataset (Students Generate)

- 30 genes
- 12 samples

```
import numpy as np

np.random.seed(1)
gene_data = np.random.randint(10, 100, size=(30, 12))
```

Each row → Gene
Each column → Sample

Tasks

1. Calculate mean expression of each gene.
2. Find top 5 highly expressed genes (based on mean).
3. Calculate mean expression per sample.
4. Identify which sample has highest total gene expression.
5. Count how many genes have average expression above overall dataset mean.
6. Normalize gene expression (Min-Max scaling).
7. Find genes whose expression is below 20 in more than 5 samples.
8. Remove low-expression genes (mean < 30) and create new dataset.
9. Calculate variance of each gene and find most stable gene (lowest variance).
10. Compare first 6 samples vs last 6 samples:
 - Calculate difference in mean expression.

Goal: Students understand biological variability + NumPy operations.

Project 2: Patient Biomarker Analysis

Dataset

- 50 patients
- 5 biomarkers (e.g., Glucose, Cholesterol, BP, Protein, Enzyme)

```
np.random.seed(2)
biomarker_data = np.random.randint(50, 200, size=(50, 5))
```

Each row → Patient
Each column → Biomarker

Tasks

1. Find average value of each biomarker.
2. Identify patient with highest overall biomarker level.
3. Find biomarker with highest variability (std deviation).
4. Detect patients whose glucose level is above average.
5. Standardize dataset using Z-score.
6. Identify potential abnormal patients:
 - Any biomarker value > mean + 2×std
7. Calculate correlation between first two biomarkers.

8. Create risk score:
 $\text{Risk Score} = 0.4 \times \text{Glucose} + 0.3 \times \text{Cholesterol} + 0.3 \times \text{BP}$
Rank patients based on risk score.
9. Remove top 5 risky patients and recompute biomarker averages.
10. Count how many patients fall above overall mean in at least 3 biomarkers.

Goal: Logical filtering + statistical thinking.

Project 3 (Optional): DNA Numeric Dataset

Assume:

- 20 DNA sequences
- Length 40

Encoded as:

A=1, T=2, G=3, C=4

```
np.random.seed(3)
dna_data = np.random.randint(1, 5, size=(20, 40))
```

Tasks

1. Count frequency of each nucleotide.
2. Compute GC content per sequence.
3. Identify sequence with highest GC content.
4. Find sequences where A appears more than 15 times.
5. Compare first 10 sequences vs last 10 sequences:
 - Average GC content difference.
6. Identify most stable nucleotide distribution.
7. Detect sequences with abnormal pattern (very high G or C).

Gene Expression CSV Analysis (Using Pandas)

File Name: gene_expression.csv

Dataset Structure

Example:

```
Gene_ID,Sample_1,Sample_2,Sample_3,Sample_4,Sample_5,Sample_6,Sample_7,Sample_8,Sample_9,Sample_10
G1,45,60,55,70,50,65,75,80,62,58
G2,30,35,40,45,38,42,50,55,48,36
G3,80,85,78,90,88,92,95,85,89,91
G4,20,25,30,22,28,26,24,23,27,29
G5,60,65,70,72,68,74,76,78,71,69
```

Students load:

```
import pandas as pd
df = pd.read_csv("gene_expression.csv")
```

Project Tasks (Moderate Level)

1. Calculate mean expression for each gene.
2. Identify top 3 highly expressed genes.
3. Find gene with lowest average expression.
4. Calculate sample-wise mean expression.
5. Remove genes with mean expression < 40 .
6. Find genes highly expressed in at least 7 samples (>70).
7. Standardize dataset using Z-score.
8. Normalize expression values (Min-Max scaling).
9. Find correlation between Sample_1 and Sample_2.
10. Detect genes with high variability (std $>$ threshold).
11. Compare first 5 samples vs last 5 samples (mean difference).
12. Sort genes by descending mean expression.
13. Count how many genes are above overall dataset mean.
14. Identify outlier expression values using boxplot logic.
15. Create summary report (mean, std, min, max).

Goal: Students think statistically, not just coding.

Project 2: Patient Clinical Dataset (CSV-Based)

File Name: patient_biomarker.csv

Dataset Structure

Example:

```
Patient_ID,Glucose,Cholesterol,BP,Protein,Enzyme
P1,120,200,130,65,80
P2,150,220,140,70,85
P3,100,180,120,60,75
P4,170,240,150,72,90
P5,110,190,125,68,78
```

Project Tasks

1. Calculate average value of each biomarker.
2. Identify patient with highest glucose level.
3. Find biomarker with highest variability.
4. Detect patients with glucose above average.
5. Create risk score:
$$\text{Risk} = 0.4 \times \text{Glucose} + 0.3 \times \text{Cholesterol} + 0.3 \times \text{BP}$$

Rank patients based on risk score.
6. Identify patients with 3 or more biomarkers above average.
7. Remove patients with enzyme level < 75 .
8. Find correlation matrix of biomarkers.
9. Detect outliers using IQR method.
10. Group patients into:
 - Normal
 - Moderate Risk

- High Risk
(based on risk score thresholds)
11. Compare top 5 risky vs bottom 5 risky patients.
 12. Find biomarker most related to high risk.
 13. Create summary statistics table.
 14. Identify patients showing abnormal pattern (any value $> \text{mean} + 2 \times \text{std}$).
 15. Sort dataset based on risk score descending.

Goal: Logical filtering + group analysis + correlation.

Project 3 (Optional): DNA GC Content Analysis

File Name: dna_sequences.csv

Example:

```
Sequence_ID,Sequence
S1,ATGCGTACGATCG
S2,GGGCCCATTAAAGC
S3,ATATATATCGCGC
S4,CGCGCGCGATATA
S5,TTTAAAGGGCCC
```

Tasks

1. Calculate GC content for each sequence.
2. Identify sequence with highest GC content.
3. Count frequency of A, T, G, C.
4. Identify sequences where GC content $> 60\%$.
5. Compare first half vs second half GC content.
6. Detect abnormal nucleotide distribution.
7. Sort sequences based on GC content.
8. Find longest sequence.
9. Create new column: AT content.
10. Create summary statistics of GC content.

Project 1: Gene Expression Visualization

Dataset: gene_expression.csv

(Assume same dataset as before – multiple samples per gene)

Tasks (Visualization + Analysis)

1. Plot average gene expression (bar chart).
2. Plot expression trend of top 5 highly expressed genes (line plot).
3. Create histogram of all expression values.
4. Plot boxplot for gene expression distribution.
5. Compare first 5 samples vs last 5 samples using bar chart.
6. Highlight gene with highest mean expression in graph.
7. Plot standard deviation of each gene.

8. Create subplot dashboard:
 - Mean expression
 - Standard deviation
 - Histogram
 - Boxplot
9. Identify outliers visually using boxplot.
10. Plot correlation trend between Sample_1 and Sample_2 using scatter plot.

Goal: Students learn biological variability through visualization.

Project 2: Patient Biomarker Visualization

Dataset: patient_biomarker.csv

Tasks

1. Plot average biomarker levels (bar chart).
2. Create histogram of glucose levels.
3. Plot scatter plot: Glucose vs Cholesterol.
4. Add regression-style trend line manually.
5. Create boxplot to detect abnormal biomarker values.
6. Highlight high-risk patients in scatter plot.
7. Compare top 5 risky vs bottom 5 risky patients visually.
8. Create multi-panel dashboard:
 - Histogram
 - Scatter plot
 - Boxplot
 - Bar chart
9. Plot correlation heatmap using Matplotlib (without seaborn).
10. Visualize risk score distribution.

Goal: Connect statistical thinking with medical interpretation.

Project 3: DNA Sequence GC Content Visualization

Dataset: dna_sequences.csv

Tasks

1. Plot GC content per sequence (bar graph).
2. Plot histogram of GC content distribution.
3. Compare GC content of first half vs second half sequences.
4. Highlight sequence with highest GC content.
5. Create pie chart showing nucleotide frequency.
6. Plot AT vs GC content comparison.
7. Visualize nucleotide count using stacked bar chart.
8. Create cumulative GC content plot.

Goal: Students visualize biological composition patterns.

Moderate-Level Interview Questions (Matplotlib Focus)

1. When should we use histogram instead of bar chart?
2. How do you control figure size?
3. How do you rotate axis labels?
4. How to highlight a specific data point?
5. How to add annotation to graph?
6. How to create multiple subplots?
7. How to improve readability of crowded plots?
8. What is alpha parameter?
9. How to save figure in high resolution?
10. How to compare two distributions visually?